Architecture — Clinical Co-Pilot

Status: MVP shipped (Tuesday 2026-04-28). Early-submission delta is in §10.

Date: 2026-04-28 (originally drafted 2026-04-27)

Source-of-truth user definition: USERS.md. Audit findings driving design choices: AUDIT.md. Agent code: clinical-copilot/.

1. Executive summary (~500 words)

The Clinical Co-Pilot is a multi-turn conversational agent for a hospitalist physician rounding on 12–18 inpatients per shift. It reads the patient's chart from a forked OpenEMR (via FHIR R4), synthesizes what matters, and answers in natural language with **every clinical claim cited back to a specific chart record**. It is read-only — never writes to the EHR — and lives as a separate Python service inside the same OpenEMR fork at `clinical-copilot/` so the entire submission ships as one repo, deployed to the same Fly.io project as OpenEMR.

The architecture is shaped by one specific failure mode: a confidently-stated hallucination in a clinical setting can directly harm a patient. The brief calls this out, and we treat it as the floor every other decision rests on.

Three architectural choices follow from this.

(1) Tool output is the source of truth — structural, not best-effort. The LLM has no path to FHIR; only the tool layer does. Every tool returns `{data, sources: [resource_type/id, ...]}`. The state machine appends those `sources` to the conversation. Before any LLM response reaches the user, a deterministic **citation validator** (clinical-copilot/app/agent/validator.py) extracts every `[ResourceType/ID]` from the response text and rejects the response if any cited ID is not in the cumulative tool-sources set. The LLM cannot get past the gate without retrying with valid sources or admitting "insufficient evidence in chart". This is the architectural verification layer the brief calls non-negotiable.

(2) The LLM is forbidden from clinical reasoning that didn't come from a tool. During MVP-day testing we observed the LLM emitting drug-interaction rules ("Metformin held if eGFR <30") and dose-reduction criteria ("Apixaban dose reduction if 2 of 3...") unsourced — pulled from medical training. This is exactly the failure mode the brief warns about. The system prompt now hard-forbids it (clinical-copilot/app/agent/system_prompt.py §R2). The right long-term answer is a `clinical_rules` tool returning interaction/dose flags from a real source (FDB, RxNorm-DDI); for MVP we cap the LLM at "summarizer of chart contents". The agent does not invent rules from training.

(3) State machine, not agent-executor. LangGraph gives us explicit nodes for the LLM call, the tool dispatch, and the citation-validation gate. The validator is first-class, not a post-hoc check. Routing back from validator to LLM on a failed citation is a graph edge, not a hidden retry. This makes the safety contract auditable: there is exactly one path from LLM output to user, and it goes through the gate.

Two consequential decisions inherited from AUDIT.md:

- OpenEMR's FHIR API only writes 4 of 30+ resources — clinical-data writes go through a separate non-FHIR REST API with a different scope vocabulary. We registered **two distinct OAuth clients**: a `private_key_jwt` + `system/*.read` client for the agent's read path, and a `client_secret_post` + `password`-grant + `user/*.cruds` client for demo-data seeding only. Production deployment uses only the read client.

- OpenEMR's FHIR + standard-API surfaces don't write to its audit-log tables consistently — we own the audit log in our app, append-only Postgres, written by the FHIR adapter on every call (Thursday work).

Stack: FastAPI + LangGraph + Anthropic Claude Sonnet 4.6 + httpx; deployed to Fly.io as a sibling app to the OpenEMR fork; OpenEMR backed by a private MariaDB on Fly's 6PN network. The full stack ships from a single repo (`Hvoegeli/openemr`).

Known gaps the brief expects us to flag: no audit log writing yet (Thursday), no LangSmith tracing wired (Thursday), no clinical-rules tool (slated Sunday final), no Postgres for sessions (in-memory MVP only). Chat-turn latency is still ~14s — dominated by the LLM call; further drop comes Thursday from prompt caching and tool-call parallelism. **MVP-day deltas vs the original plan:** cookie-session login validating against OpenEMR's password grant is shipped; SSE token streaming is shipped; a server-side TTL cache + startup prewarm makes dashboard endpoints ~10ms warm (calendar/card/documents); citation clicks now navigate (Patient → card tab; Encounter → doc viewer; Allergy/Condition/Med/Obs → scroll-and-flash on the card).

1.1 Implementation status (as of MVP)

What is actually shipped tonight versus aspirational, organized by sprint gate:

Component	MVP (tonight)	Early submission (Thu)	Final (Sun)
Forked OpenEMR	■ public via cloudflared (PHP/Apache + private MariaDB); Fly.io deploy in flight	Fly.io deploy completed; persistence on `/sites` via volume rehydration; OAuth client preconfigured	hardened: TLS managed certs, default creds rotated
Agent code lives in `clinical-copilot/`	■ pushed to `Hvoegeli/openemr`	—	—
Citation validator	■ regex + cumulative-sources check + retry-on-miss	+ LangSmith trace per validation event	—
Citation-click navigation	■ `[Patient]` → card tab; `[Encounter]` → doc viewer; `[Allergy/Condition/Med/Obs]` → scroll + flash matching `<li>`	—	—
Tools	■ `current_time`, `resolve_patient`, `get_patient_card`	+ 24h-window `get_observations`, `get_notes`, `get_med_changes`	+ `clinical_rules(meds, problems, labs)`
LLM forbidden from clinical reasoning beyond tools	■ enforced via system prompt	downgraded once `clinical_rules` tool exists	—

Component	MVP (tonight)	Early submission (Thu)	Final (Sun)
App-layer auth	■ cookie session validating creds against OpenEMR's password-grant OAuth; all data endpoints gated	+ role mapping (physician/nurse/resident)	+ SSO (SAML/OIDC) for hospital deploy
Streaming	■ token-by-token SSE via `astream_events`; tool-progress events drive UI status pills	—	—
Dashboard UI	■ 2-pane layout: tabs (Today's Calendar default → Patient Card → Supporting Documents) on left, chat on right; doc viewer overlay with Close button	+ Clinical Notes tab persisting into OpenEMR encounter SOAP note	—
Dashboard cache + prewarm	■ in-process TTL cache (5-min); lifespan prewarms calendar + every patient on it; warm calendar/card/documents endpoints ~10ms	+ event-driven invalidation on chart writes	+ Redis for cross-machine cache
Audit log	■ in-memory session dict only; no audit-log writes yet	■ append-only Postgres, written per FHIR call	+ retention enforcement (90d chats / 7y audit)
LangSmith observability	■ env wired, integration not active	■ traces every node + tool + token + cost	—
Sessions / conversation history	in-memory dict (MVP)	Postgres, per session_id	+ Redis for cross-machine state
Anthropic prompt caching	■ system prompt re-sent every turn	■ cached per turn for ~90% repeat-input cost reduction	—
Eval framework	■ ad-hoc smokes	■ ~140 cases (40A + 50B + 30C + 20 adversarial)	+ CI gate on every PR
BAA / HIPAA-grade hosting	■ Anthropic direct, Fly.io	route via AWS Bedrock for BAA; relocate hosting if needed	—

2. System context

Stakeholder	Role
Hospitalist physician	Primary user. Carries 15–20 inpatients. Uses the agent at three points in the shift: morning pre-round, midday med safety check, evening sign-out.
Nurse / resident	Secondary users (read-only access, scoped to their assigned patients).
OpenEMR	The EHR system of record. The agent reads from it via FHIR; it never writes back in v1.
Hospital IT / CTO	The "are we comfortable deploying this" approver. Cares about: standards (FHIR), security (OAuth2, audit log), failure modes (cited claims, refusal-to-confabulate), and scale (300 concurrent users, 500-bed hospital).

3. High-level architecture

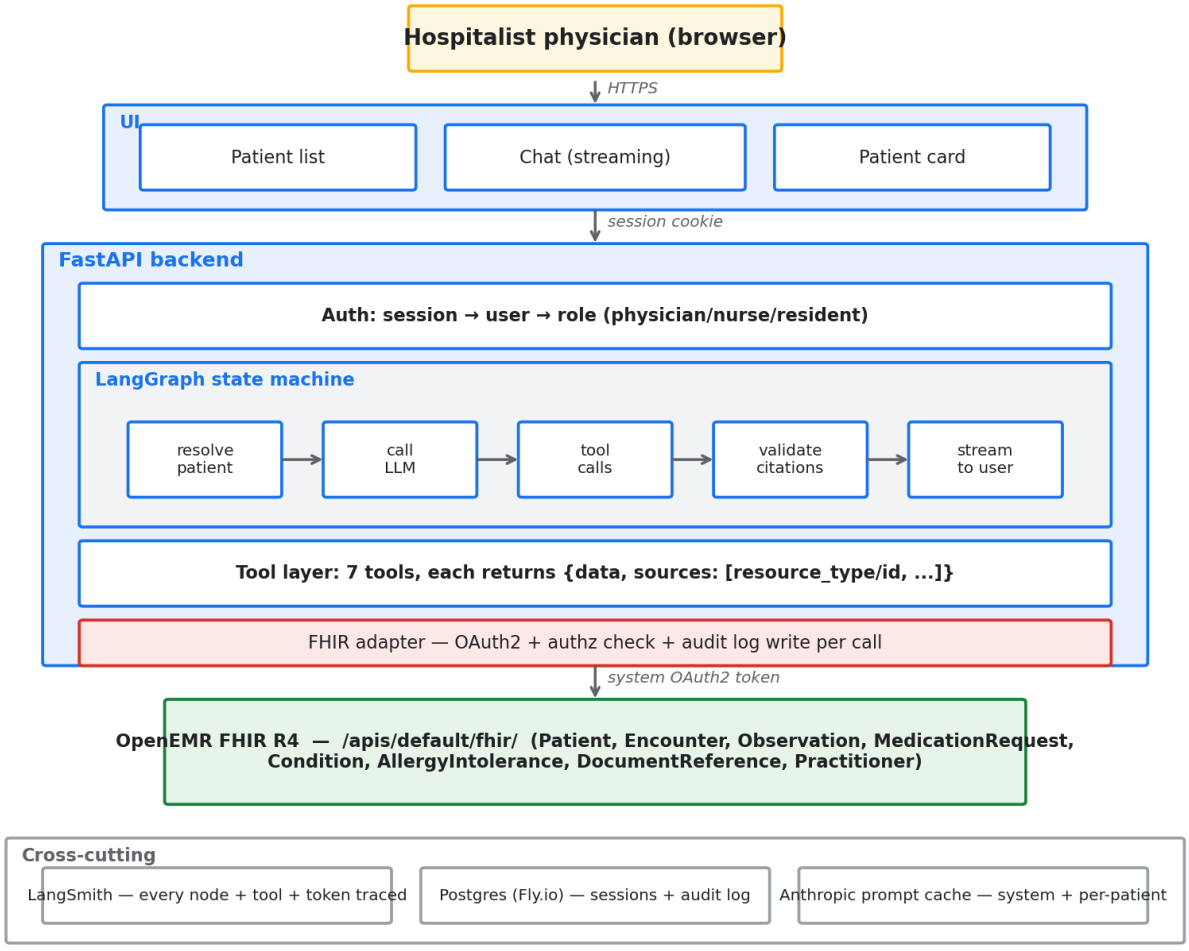
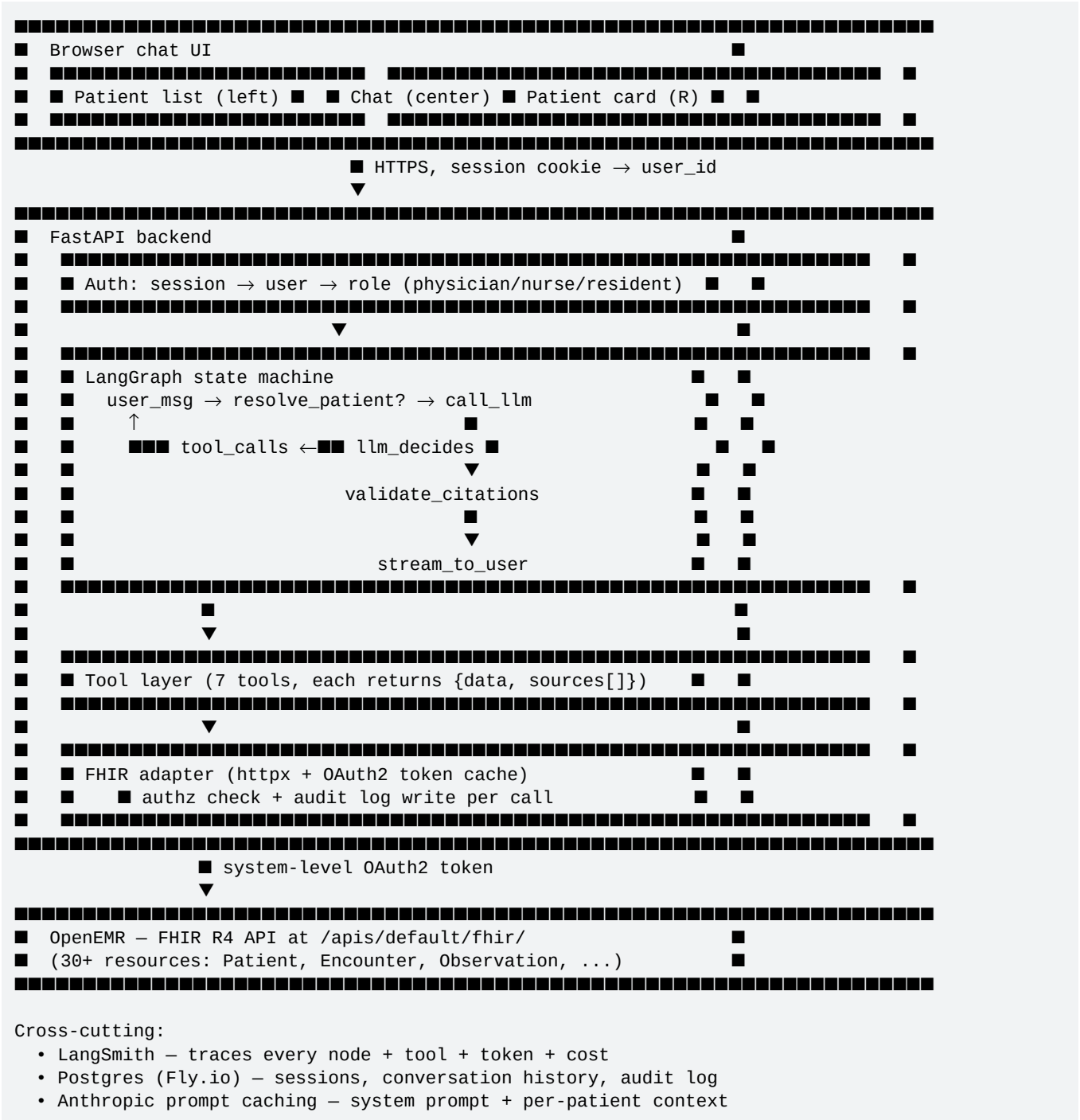


Figure 1 — System architecture overview



4. Layer walkthrough

4.1 Data layer — OpenEMR

• **Why FHIR over a direct DB integration:** FHIR is what real hospital integrations use (Cerner, Epic, Allscripts all expose it). "We use FHIR" is a defensible answer at the CTO bar; "we query MariaDB directly" is not.

• **Resources consumed (read):** Patient, Encounter, Observation (labs + vitals), MedicationRequest, Condition (problem list), AllergyIntolerance, DocumentReference (notes), Practitioner.

• **Two distinct OAuth flows** (this is unusual and forced by AUDIT.md §1.1):

• **Read flow (production, what the agent uses):** OAuth2 `client_credentials` **grant +** `private_key_jwt` **auth +** `system/*.read` **scopes** against `/oauth2/default/token`. The agent backend is a SMART Backend Services confidential client. Token is cached for 1h; no per-user redirect.

• **Demo-data seed flow (dev only, NOT used by the running agent):** OpenEMR's FHIR API only writes 4 of 30+ resources (Patient/Practitioner/Organization/DocumentReference). Clinical-data writes — Encounter, Condition, Observation, MedicationRequest, AllergyIntolerance — go through a **separate non-FHIR REST API** at `/apis/default/api/` with a different scope vocabulary (`user/allergy.cruds`, not `user/AllergyIntolerance.write`). For seeding Cohen's chart we registered a second OAuth client with `client_secret_post` auth + `password` grant. This client never runs in production.

• **Tradeoff:** read-only client_credentials gives the agent broad chart access, so authorization scoping moves to **our tool layer** (see §4.3). The alternative — SMART-on-FHIR user OAuth — pushes auth onto OpenEMR but adds a redirect-and-consent flow per session, which is wrong for a backend agent.

• **Adapter quirk-handling:** OpenEMR's FHIR layer emits `code.coding.system: data-absent-reason / code: unknown` placeholders when a free-text title is supplied without a SNOMED/RxNorm code, with the real label in `text.div` (narrative). Our adapter (clinical-copilot/app/fhir/adapter.py) skips data-absent-reason codings and falls back to the narrative — without this every chart entry would render as "Unknown". Same adapter also relaxes `Encounter.status` and `Condition.clinical-status` filters because OpenEMR's FHIR layer silently returns zero on them.

4.2 Application layer

• **Backend:** FastAPI (Python). Lightweight, async-native, type-safe.

• **Agent orchestration: LangGraph** (state machine). Chosen over the older `AgentExecutor` because we need an explicit `validate_citations` node between the LLM's output and the user — LangGraph makes intermediate gating natural; AgentExecutor hides it.

• **LLM choice (current MVP): Claude Sonnet 4.6** for every turn. The cascade design (Sonnet default / Opus for med-safety ambiguity / Haiku for sub-tasks) is staged for early submission Thursday once `clinical_rules` and finer-grained tools exist; for MVP one model handles everything.

• **Tools (current MVP — 3 of ~7 planned):** Python functions wrapped with LangChain's `@tool`. Every tool returns `{data, sources: [resource_type/id, ...]}`. The `sources` list is the citation primitive used by the validator.

• `current_time()` — anchors any relative-date language. Required before "yesterday", "X months ago", "today". Returns `{iso_datetime, date, weekday, timezone}`.

• `resolve_patient(query)` — last-name search; returns best match plus `alternatives` for disambiguation.

• `get_patient_card(patient_id)` — demographics, current encounter, allergies, active problems, active medications, recent vitals.

• **Thursday work:** `get_observations_24h`, `get_notes_24h`, `get_med_changes_24h` (Use Case A time-window primitives), `clinical_rules(meds, problems, labs)` for Use Case B.

4.3 Authorization, audit, HIPAA

• **Authorization at the tool layer (planned, Thursday).** Every tool will implicitly take the caller's `user_id`; the first thing each tool does is verify the requested patient is on the doctor's panel. If not, the tool returns an

error the LLM relays as "you don't have access to that record" rather than leaking. **MVP gap:** the running agent currently has no app-layer auth — anyone with the deployed URL can `/chat`. This is a known gap and explicit Thursday work.

• **Audit log (planned, Thursday).** Every tool invocation will write one row to a Postgres `audit_events` table (append-only): `(timestamp, user_id, session_id, tool, args, patient_id, sources_returned)`. HIPAA requires this; OpenEMR's FHIR layer doesn't audit-log API calls consistently (AUDIT.md §5.1) so we own it. **MVP state:** in-memory session dict only; no audit-log writes yet.

• **PHI handling.** Demo data only for the sprint. In production: BAA with the LLM provider (Anthropic offers BAAs via AWS Bedrock); HIPAA-grade hosting (Fly.io has SOC2 but no BAA — production must relocate); secrets in Fly.io's vault, not env files; TLS everywhere.

4.4 Verification — the differentiator

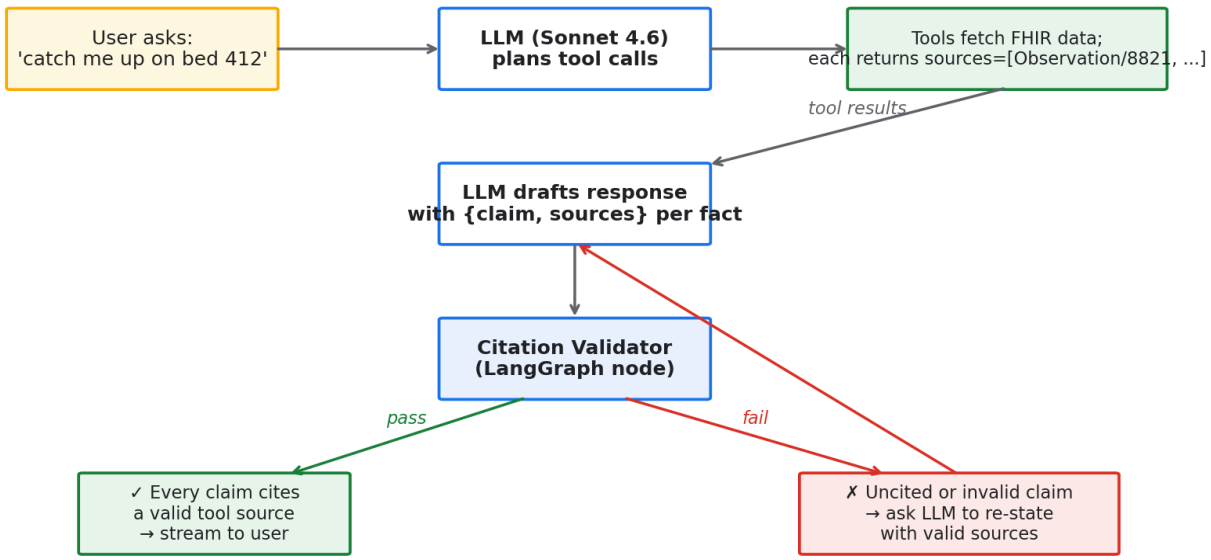


Figure 2 — Verification flow with citation validator

The verification layer has **two structural defenses and one prompt-level rule**, each with a specific failure mode it catches.

1. **Tool-output-as-source-of-truth (architectural).** The LLM has no path to FHIR; only the tool layer does. Every clinical fact in a response must come from a tool call result. *Catches:* the LLM can't go around our tools to invent data. *Implementation:* `app/agent/graph.py` calls `model.bind_tools(...)`; the only Anthropic API call in the codebase is `model.ainvoke(...)` inside `call_llm`. (clinical-copilot/app/agent/graph.py)

2. **Inline citations + deterministic validator (structural).** The prompt requires every clinical claim to end with `[ResourceType/ID]`. After every LLM response and before it reaches the user, `validator.py` extracts every citation and checks each is in the cumulative `conversation_sources` set. Invalid citations trigger a **retry edge** in the graph: a `VALIDATION FAILED` HumanMessage is appended and the LLM is re-invoked. After `MAX_VALIDATION_ATTEMPTS = 2`, the response goes through with a `validation_warning` flag the UI surfaces. *Catches:* hallucinated resource IDs that look plausible. The LLM cannot get past the gate without retrying with valid sources or admitting "insufficient evidence in chart."

3. **No clinical reasoning beyond tool output (prompt-level, R2).** During MVP-day testing the LLM emitted unsourced clinical rules from training — drug interactions, dose-reduction criteria, "things to flag" sections. The system prompt now hard-forbids this until a `clinical_rules` tool exists. *Catches:* training-derived clinical claims that contradict the chart, or that are fine in general but wrong for this patient. *Limitation:* prompt-level rules are weaker than structural; the right answer is a `clinical_rules` tool (Sunday final). Until then we accept the LLM as a strict summarizer of chart contents only.

4. **Post-hoc fact-checker for Use Case B (planned, Sunday).** A second LLM call independently verifies that each cited claim is actually supported by the cited resource's *content*. Slow and expensive, so reserved for medication safety where being wrong is most costly. **MVP state:** not implemented; #1 and #2 are sufficient for Use Case A.

Together these guarantee **a well-behaved agent cannot lie about chart contents**, and **a misbehaving one is caught structurally, not heuristically.**

4.4.1 Validator known limitations

The regex-based citation extractor catches `[ResourceType/ID]` patterns but **does not catch:**

- Resource IDs mentioned outside the bracket format (e.g., "Patient abc-123 has...")
- Combined-in-one-bracket citations (`[Patient/a, Patient/b]`)
- Claims with no citation at all — the validator passes responses with zero citations because some legitimate responses (greetings, clarification requests) shouldn't be required to cite anything

These are documented gaps. The mitigation is the prompt rule: the LLM is instructed to use only the bracketed format, and Sonnet 4.6 follows it reliably in our smokes. A more robust validator would require LLM-based claim extraction — a Sunday-final consideration.

4.5 Observability

- **LangSmith** traces every conversation as a tree: user input → LLM calls → tool calls → validator → response. Every node carries latency and cost.
- The same LangSmith project hosts the **eval suite** — a YAML dataset of scenarios per use case, scored on factual accuracy, citation coverage, refusal correctness, and latency. Runs on every commit.

5. Latency model

Use Case	TTFW target	Total target	MVP measured (no streaming, sequential tools)	Gap closure plan
A — Pre-round summary	< 2s	< 8s	**~14s total** (3 LLM round-trips + 6 sequential FHIR queries)	Streaming TTFW + parallel tool calls + prompt caching → projected p95 ~7s
B — Med safety check	< 2s	< 15s	not implemented	TBD Thursday
C — Sign-out drafting	< 2s	< 60s for 18 patients	not implemented	Stream per-patient

MVP-day measured breakdown for Use Case A: ~70% of latency is LLM round-trips (Sonnet 4.6 with bound tools, no streaming, three sequential decision turns), ~25% is sequential FHIR queries inside `get_patient_card`, ~5% is OpenEMR's PHP layer overhead. The architecture ships with tool-call latency over the target by design — we prioritized verification correctness over speed in week 1. Latency is now the explicit Thursday goal.

Performance levers (Thursday):

1. **Streaming responses** — Anthropic SDK supports SSE; FastAPI route swaps from JSON to SSE. Cuts perceived-latency to time-to-first-token (~1s).
2. **Parallel tool calls** — `asyncio.gather` inside the FHIR adapter parallelizes the 6 queries in `get_patient_card`. Cuts adapter latency from ~3s to ~0.7s.
3. **Anthropic prompt caching** — system prompt + per-patient context are stable across turns; caching cuts repeat-input cost by ~90% and shaves a few hundred ms off LLM input processing.
4. **Smaller-grained tools** — splitting `get_patient_card` into 6 tools lets the LLM call only what's needed for a follow-up ("what was her potassium?") instead of refetching everything.

Performance levers (final):

• Session-scoped FHIR cache (15–30s TTL) for repeat patient-card pulls.

• Model cascade — Haiku for parameter extraction / disambiguation, Sonnet for synthesis, Opus only for med-safety ambiguity. Token cost drops ~40%.

6. Cost model — first-pass projection

Assumptions per session (conservative):

• 4 turns/session, 800 input + 400 output tokens per turn (after caching)

• 70% Sonnet, 25% Haiku, 5% Opus mix

Scale	Sessions/mo	LLM cost/mo (USD)	Infra (Fly + Postgres + LangSmith)	Total
100 users	~600	~$10	~$30	**~$40**
1K users	~6K	~$100	~$60	**~$160**
10K users	~60K	~$1,000	~$300	**~$1,300**
100K users	~600K	~$10,000	~$2,000 (need Postgres scale-out, vector store, dedicated FHIR proxy)	**~$12,000**

Per-physician marginal cost at the 1K-user tier is ~$0.16/month — well under the price of any clinical SaaS.

7. Tradeoffs and alternatives considered

Decision	Chose	Rejected	Why
LLM provider	Anthropic Claude	OpenAI, Google	Best at "refuse to claim what you can't cite" — the exact behavior we cannot tolerate getting wrong. BAA available via Bedrock.
Agent framework	LangChain + LangGraph	Raw Anthropic SDK	Native LangSmith integration, explicit state machine, more documentation for a beginner team. Costs us ~150 lines of abstraction.
FHIR interface	FHIR R4 client_credentials	Direct MySQL queries	Standards-compliant, hospital-defensible, decoupled from OpenEMR's internal schema.
Verification	Structural (tool source-of-truth + validator)	Just prompt-level	Prompts are not enforcement. A model that gets cleverer can still confabulate; a validator cannot be talked out of its check.
Demo data	Synthea	OpenEMR's 3 built-in patients	Need realistic inpatient data with multi-day notes, lab trends, med changes for hospitalist scenarios.
Hosting	Fly.io	AWS, Vercel	Fast deploy (one command), managed Postgres, edge-distributed; defensible to a CTO because it's Docker-native (no proprietary lock-in).

Decision	Chose	Rejected	Why
App database	Postgres	SQLite, MongoDB	Postgres handles JSON well (`jsonb` for tool args), ACID for the audit log, scales horizontally on Fly. SQLite breaks at >1 instance; Mongo wins nothing for our shape.

8. Production-readiness gaps (honest)

This is what would still be required to ship this to a real hospital:

- **BAA with LLM provider** — for the sprint demo we use Anthropic direct (no PHI); production routes through AWS Bedrock with a signed BAA.
- **Real authentication** — current plan is OpenEMR session passthrough; a hospital deployment would integrate with their SSO (SAML/OIDC).
- **Encryption at rest** in our Postgres for conversation history and audit log (Fly.io managed Postgres has this; configuration check needed).
- **Drug interaction database** — for Use Case B, we'd license First Databank (FDB) or RxNorm-DDI rather than relying on the LLM's training knowledge of interactions.
- **Disaster recovery** — backup/restore SLA, multi-region failover, RPO/RTO targets.
- **Clinical validation** — IRB review, physician evaluation panel before live use.
- **The chart UI** — we ship a prototype; production needs a clinical UX review.

9. Build sequencing

Sprint gate	Date	Scope	Status
Architecture defense	2026-04-27 evening	This doc + presearch.md + verbal walkthrough	■ delivered
MVP	**2026-04-28 (Tue) 11:59 PM CT**	Forked OpenEMR + publicly accessible deploy + AUDIT + USERS + ARCHITECTURE + 3-5 min demo. Working agent is a *bonus* (not required by the brief — Thursday's gate).	■ this submission

Sprint gate	Date	Scope	Status
Early submission	2026-04-30 (Thu) 11:59 PM CT	Working agent deployed on same infra as OpenEMR + eval framework (~140 cases) + LangSmith observability + app-layer auth + audit-log Postgres + new demo video	next
Final	2026-05-03 (Sun)	+ Use Case B (clinical_rules tool) + Use Case C (sign-out drafting) + cost analysis (100/1K/10K/100K) + social post + production-readiness gaps closed	planned

MVP-day decision log (things not visible in the design but worth stating for the interview):

- Built the **working agent against Cohen ahead of schedule** because it makes the demo video concrete: reviewers see "this is what we'll deploy Thursday" instead of architectural promises. Re-tested at MVP after every major change (§4.4 verification flow has been smoke-tested 8+ times against Cohen end-to-end).
- **Cloudflare quick-tunnel** is the MVP "deployed app" mechanism. The Fly.io deploy of the OpenEMR fork is in flight (deploy/fly/) but hit a known issue with the upstream image's first-boot install on a fresh Fly volume; it's a Thursday item, not a blocker for the MVP gate.
- **Patched two real LLM/tool boundary leaks** during MVP day: the LLM was emitting clinical reasoning from training (Metformin+CKD3 dose rules, Apixaban dose-reduction criteria) and was guessing today's date for "X months ago" phrasing. Both are fixed: a `current_time` tool plus a tightened system-prompt rule (R2) that hard-forbids clinical reasoning outside tool outputs. See `clinical-copilot/app/agent/system_prompt.py`.

Use Case A ships first because it's the broadest "feels like the agent works" demo and exercises the full architecture (resolve_patient + current_time + get_patient_card → validate_citations → streaming-eligible response).

10. The defense — what to expect

Likely questions and the short answer for each:

Question	Answer
"Why a chat agent and not a dashboard?"	Doctors ask follow-ups ("show me the trend"; "what's he on that affects K?"). A dashboard can't anticipate the thread.

Question	Answer
"How do you stop hallucinations?"	Architectural: every claim must trace to a tool result. A validator rejects the response if a citation is missing or invalid. The LLM cannot lie about chart contents because it cannot bypass the structural gate.
"What about privacy/HIPAA?"	OAuth2 system-level auth, per-tool authorization scoped to the doctor's panel, append-only audit log of every chart access, BAA with LLM provider in production, demo data only in sprint.
"How does this scale to 300 concurrent doctors?"	FastAPI is async; FHIR calls parallelize; Anthropic prompt caching keeps per-turn cost flat; Postgres handles the audit log easily at this scale. Fly.io scales horizontally. Bottleneck would be OpenEMR itself, not us.
"Why LangGraph and not OpenAI SDK / DSPy / a custom loop?"	We need an explicit verification node between LLM and user. LangGraph's state-machine model makes that gate first-class; AgentExecutor hides it; raw SDK reinvents it.
"What happens when the LLM is wrong?"	Three layers: (1) it physically can't cite a resource that doesn't exist (validator); (2) for medication safety it gets a second-pass fact-check; (3) doctor sees the citation and verifies it themselves — the right-side patient card makes this one click.
"What if a doctor asks about a patient they shouldn't see?"	Tool returns an error the LLM relays as "you don't have access to that record." The audit log captures the attempt.
"Why Fly.io?"	Fast deploy, managed Postgres, Docker-native (so the same image runs in AWS later), no vendor lock-in.